FMX Academy

INFORMAÇÕES			
MÓDULO		**AULA**	**INSTRUTOR**
07	**ASP.NET Core Web API – Avançado - Padrões de Projeto, Arquitetura Limpa e Alta Performance**	**06 - Arquitetura de Alta Performance - Continuação**	**Guilherme Paracatu**

1. Prática – Implementando a Stack de Leitura

🛠 **Exercício de Fixação**

Desafio: Relatório de Estoque A diretoria precisa de um endpoint que mostre quantos veículos existem por Modelo. Isso não existe na tabela, é um dado calculado.

1. Crie o DTO: RelatorioModeloDTO (Propriedades: NomeModelo, Quantidade).
2. Implemente um método na Query usando SqlQuery com GROUP BY.
3. Crie o endpoint [HttpGet("relatorio-estoque")] na Controller.

Diagrama da Arquitetura:

- A requisição **GET** pega um atalho (Controller -> Query -> Banco).
- A requisição **POST** pega o caminho seguro (Controller -> Bus -> Handler -> Domain -> Repo -> Banco).
- Isso é **CQRS**: caminhos diferentes para necessidades diferentes.

2. Desafio Avançado: Mestre-Detalhe com SQL Puro

O Cenário

Queremos listar todas as **Marcas**, e dentro de cada marca, uma lista de seus **Veículos**.

- **Problema:** O SQL não retorna listas dentro de listas. Ele retorna linhas repetidas.
- **Solução:** Buscamos os dados "achatados" (Flat) via SqlQuery e usamos o LINQ (GroupBy) na memória para montar a árvore.

Passo 1: Os DTOs de Retorno (A Árvore)

Local: ITB.Application/DTOs/MarcaComVeiculosDTO.cs

Código C#

```
namespace ITB.Application.DTOs;

// 1. O Filho (Simplificado)
public class VeiculoSimplesDTO
{
    public int Id { get; set; }
    public string Placa { get; set; } = string.Empty;
    public string Modelo { get; set; } = string.Empty;
    public int Ano { get; set; }
}

// 2. O Pai (Com a Lista)
public class MarcaComVeiculosDTO
{
    public int Id { get; set; }
    public string Nome { get; set; } = string.Empty;
    public bool Ativo { get; set; }
    public List<VeiculoSimplesDTO> Veiculos { get; set; } = new();
}
```

Passo 2: O DTO Intermediário (O "Flat")

Representa o retorno "achatado" do banco.

Nota: Graças à biblioteca de convenção, o EF vai mapear marca_id do SQL para MarcaId do C# automaticamente.

Local: ITB.Infrastructure/Queries/DTOs/MarcaVeiculoFlatDTO.cs

Código C#

```
public class MarcaVeiculoFlatDTO
{
    // Dados da Marca
    public int MarcaId { get; set; }
    public string MarcaNome { get; set; } = string.Empty;
    public bool MarcaAtivo { get; set; }

    // Dados do Veículo (Podem ser nulos por causa do LEFT JOIN)
    public int? VeiculoId { get; set; }
    public string? VeiculoPlaca { get; set; }
    public string? ModeloNome { get; set; }
    public int? VeiculoAno { get; set; }
}
```

Passo 3: A Query com Transformação (Look-Up Pattern)

Local: ITB.Infrastructure/Queries/MarcaQuery.cs

Código C#

```
public async Task<IEnumerable<MarcaComVeiculosDTO>> ObterMarcasComVeiculos()
{
    // 1. SQL Otimizado para PostgreSQL (Snake Case)
    // - Tabelas e colunas em minúsculo
    // - Aliases com underline (ex: marca_id) para bater com a convenção

    var query = _context.Database.SqlQuery<MarcaVeiculoFlatDTO>($@"
        SELECT
            m.id      AS marca_id,
            m.nome    AS marca_nome,
            m.ativo   AS marca_ativo,
            v.id      AS veiculo_id,
```

```
            v.placa   AS veiculo_placa,
            v.ano     AS veiculo_ano,
            mod.nome  AS modelo_nome
        FROM marcas m
        LEFT JOIN modelos mod ON mod.marca_id = m.id
        LEFT JOIN veiculos v ON v.modelo_id = mod.id
    ");

    // 2. Executa a query "Flat"
    // O EF Core + NamingConventions mapeia: "marca_id" (Banco) -> "MarcaId" (DTO)
    var dadosPlanos = await query.ToListAsync();

    // 3. Transformação em Memória (Group By)
    // Agrupamos pelo ID da Marca para eliminar a repetição
    var resultadoHierarquico = dadosPlanos
        .GroupBy(x => x.MarcaId)
        .Select(grupo => new MarcaComVeiculosDTO
        {
            Id = grupo.Key,
            // Pegamos os dados do primeiro item do grupo (dados da marca são
repetidos)
            Nome = grupo.First().MarcaNome,
            Ativo = grupo.First().MarcaAtivo,

            // 4. Projetamos a lista de filhos
            // Filtramos onde VeiculoId != null (ignora marcas sem carros)
            Veiculos = grupo
                .Where(x => x.VeiculoId.HasValue)
                .Select(x => new VeiculoSimplesDTO
                {
                    Id = x.VeiculoId!.Value,
```

```
                    Placa = x.VeiculoPlaca!,
                    Modelo = x.ModeloNome!,
                    Ano = x.VeiculoAno!.Value
                }).ToList()
        });

    return resultadoHierarquico;
}
```

Passo 4: Explicando Trade-offs (Trocas)

- **O Jeito Preguiçoso (Lazy Loading):** Fazer um loop nas marcas e buscar os veículos de cada uma.
  - *Resultado:* Problema N+1 (milhares de queries). Lento.
- **O Jeito EF Core (Includes):** Usar .Include().
  - *Resultado:* Fácil de escrever, mas traz overhead de rastreamento (tracking) quando não é usado corretamente.
- **O Jeito SQL Raw (Este exemplo):**
  - *Resultado:* **Performance Máxima.**

🕵 **Dica:**

"Pessoal, reparem que criamos o MarcaVeiculoFlatDTO. O SQL não sabe o que é um Objeto dentro de outro Objeto, ele só conhece linhas e colunas.

O trabalho do Arquiteto de Software é justamente esse: ir ao banco buscar as peças soltas (Flat DTO) e montar o Lego (Hierarquia) dentro da aplicação."

🛠️ Otimização Extrema EF Core com Projeção (.Select)

🎓 O Que é Projeção (.Select)?

Imagine que você vai ao supermercado comprar apenas **pão e leite**.

1. **Sem Projeção (ToList direto):** Você coloca **o supermercado inteiro** no carrinho, leva para o caixa, paga por tudo, leva para casa, e lá na sua cozinha você joga tudo fora e fica só com o pão e o leite.

2. **Com Projeção (.Select):** Você pega apenas o pão e o leite na prateleira, paga só por eles e leva para casa.

No EF Core, **Projeção** é o ato de transformar os dados **ainda dentro do Banco de Dados**, trazendo para a memória (C#) apenas o que você realmente precisa.

🆚 O Comparativo: Jeito Errado vs. Jeito Certo

Vamos supor que a tabela Veiculos tem 50 colunas (incluindo uma foto do carro em Base64 que pesa 2MB).

❌ O Jeito "Gastão" (Sem Projeção)

Código C#

```
// 1. O EF vai no banco e faz um "SELECT *" (Traz as 50 colunas, inclusive a foto pesada)
var veiculos = await _context.Veiculos.ToListAsync();


// 2. O C# recebe tudo na memória (RAM explode 💥)
// 3. Depois, você converte para DTO
var dtos = veiculos.Select(v => new VeiculoDTO {
    Id = v.Id,
    Placa = v.Placa
}).ToList();
```

- **Problema:** Você tráfegou megabytes de dados pela rede desnecessariamente.

✅ O Jeito "Econômico" (Com Projeção)

Código C#

```
// 1. O EF entende que você só quer 2 campos.
var dtos = await _context.Veiculos
    .Select(v => new VeiculoDTO // <--- ISSO É A PROJEÇÃO
    {
        Id = v.Id,
        Placa = v.Placa
    })
    .ToListAsync();
```

🕵️ O Que Acontece no SQL (Por Baixo dos Panos)?

SQL Gerado SEM Projeção:

```
SELECT id, placa, modelo, chassi, cor, ano, observacao, foto_blob, data_cadastro,
usuario_alteracao...
FROM veículos
```

SQL Gerado COM Projeção (.Select):

```
SELECT id, placa
FROM veiculos
```

🚀 Por que usar Projeção no EF Core 8?

1. **Performance de Rede:** Traz apenas os bytes necessários. Se a tabela tem 1GB de dados mas você só pediu os IDs, virão apenas alguns KBs.
2. **Performance de Memória:** O C# não precisa instanciar a Entidade Veiculo inteira (que é pesada). Ele cria direto o VeiculoDTO (que é leve).
3. **No-Tracking Automático:** Quando você usa .Select para criar um DTO, o EF Core **não rastreia** as mudanças (pois DTO não é Entidade). Isso é muito mais rápido do que um Include normal.
4. **Resolução de N+1:** O EF Core 8 é inteligente o suficiente para traduzir projeções complexas (como listar Veículos dentro de Marcas) em um único SQL otimizado, evitando loops no banco.

No contexto de **CQRS**, a Projeção é a melhor amiga da Query, pois permite preencher os DTOs de leitura diretamente, sem passar pelas Entidades de Domínio.

Ao usar o .Select() diretamente na consulta LINQ, o EF Core é inteligente o suficiente para entender que precisa fazer os JOINs, então você **nem precisa escrever o .Include() e nem o .AsNoTracking()** (pois projeções são No-Tracking por padrão).

Isso é ainda mais rápido porque só busca as colunas que você pediu (ex: ignora colunas pesadas como Foto ou Observacao se elas não estiverem no DTO).

Versão Otimizada Mestre Detalhe (Projection):

Código C#

```
public async Task<IEnumerable<MarcaComVeiculosDTO>> ObterMarcasComVeiculosProjecao()
{
    // O EF Core vai gerar um SQL que busca APENAS essas colunas específicas.
    // É o melhor da performance com LINQ.
    return await _context.Marcas
        .Select(m => new MarcaComVeiculosDTO
        {
            Id = m.Id,
            Nome = m.Nome,
            Ativo = m.Ativo,

            // O EF entende que isso é um JOIN e monta a query sozinho
            Veiculos = m.Veiculos.Select(v => new VeiculoSimplesDTO
            {
                Id = v.Id,
                Placa = v.Placa,
                Ano = v.Ano,
                Modelo = v.Modelo.Nome // Já navega e busca o nome
            }).ToList()
        })
```

```
        .ToListAsync();
}
```

🧠 **Resumo:**

1. **AsNoTracking() + Include():** Traz a tabela inteira (todas as colunas), mas não vigia mudanças. Ótimo quando você precisa de quase todos os dados.
2. **Select() (Projeção):** Traz apenas as colunas que você escolher. É o campeão de performance no LINQ, pois economiza tráfego de rede.

Ambas as abordagens são perfeitas para a **Stack de Leitura (Query)** do CQRS!

Quando usar Projeção? **Você deve usar Projeção (.Select) como padrão em 95% dos casos.**

A diferença de performance entre uma **Projeção LINQ bem feita** e um **SQL Puro (SqlQuery)** no EF Core 8 é **insignificante** para a grande maioria das aplicações comerciais.

Aqui está a explicação técnica, separando o "Mito" da "Realidade":

1. Por que a Projeção (.Select) deve ser a sua primeira escolha?

O ganho de produtividade e segurança do LINQ supera, e muito, os nanosegundos que você ganharia escrevendo SQL na mão.

1. **Type Safety (Segurança de Tipo):**
   - **LINQ:** Se você mudar o nome da propriedade Placa para NumeroPlaca na classe, o Visual Studio avisa o erro de compilação na hora.
   - **SQL Puro:** O erro só vai estourar quando o cliente rodar a tela (Runtime Error), porque o SQL é uma string mágica ("SELECT Placa...").
2. **Refatoração Automática:**
   - Renomear propriedades ou mudar tipos de dados no C# atualiza a query LINQ automaticamente. No SQL Puro, você tem que caçar todas as strings no projeto (Ctrl+F) e rezar para não esquecer nenhuma.
3. **Independência de Banco:**
   - O LINQ gera o SQL correto seja para SQL Server, PostgreSQL ou Oracle. Se você escrever SQL na mão (SELECT TOP 10...), vai quebrar se mudar para um banco que usa LIMIT 10.

2. Então, quando usar SQL Puro (SqlQuery ou Dapper)?

Se o LINQ é tão bom, por que ensinamos o SQL Puro na aula de CQRS? Porque existem **3 cenários** onde o LINQ "pede água":

A. Queries Excessivamente Complexas

O EF Core 8 é genial, mas às vezes ele se perde. Se você tiver muitos Group By, sub-queries, Union e funções de janela (ROW_NUMBER, RANK), o SQL gerado pelo LINQ pode ficar **muito feio** e lento.

Nesse caso, você escreve o SQL na mão para garantir que o banco use os índices corretos.

B. CTEs e Recursos Específicos (Common Table Expressions)

Até pouco tempo, o EF Core não suportava CTEs (aquelas queries com WITH TabelaAuxiliar AS...) de forma nativa e fácil. Para relatórios hierárquicos ou recursivos, o SQL Puro é mandatório.

C. A "Tradução" do LINQ (Overhead)

Toda vez que você roda um LINQ, o .NET precisa "traduzir" aquela expressão C# para SQL.

- **A Realidade:** O EF Core faz **cache** dessa tradução. A primeira execução demora 10ms a mais, as próximas são instantâneas.
- **O Extremo:** Se você tiver uma query que roda 1 milhão de vezes por minuto, esses micro-segundos de tradução contam. Aí o SQL Puro ganha.

📊 **Comparativo Realista (Benchmark Mental)**

Imagine uma consulta simples de listagem de veículos:

Método	**Tempo de Execução (Exemplo)**	**Manutenibilidade**	**Segurança**
LINQ Projection (.Select)	55ms	⭐⭐⭐⭐⭐ (Excelente)	⭐⭐⭐⭐⭐ (Total)
EF Core (SqlQuery)	50ms	⭐⭐⭐ (Média)	⭐⭐ (Baixa - String Mágica)
Dapper	48ms	⭐⭐ (Baixa - Gestão de Conexão)	⭐⭐ (Baixa)

Conclusão: Vale a pena perder a segurança de tipo e a facilidade de manutenção para ganhar 5ms ou 7ms? **Geralmente, não.**

👩‍⚖️ O Veredito (A Regra de Ouro)

1. **Nível 1 (Padrão):** Use **LINQ com Projeção (.Select).**
    - É rápido (traz apenas as colunas necessárias).
    - É seguro (compilação).
    - É limpo.
2. **Nível 2 (Otimização):** Use **SQL Puro (SqlQuery)** APENAS SE:
    - O LINQ estiver gerando um SQL lento (verifique no Profiler).
    - Você precisar de um recurso que o LINQ não suporta (ex: HINTS de banco, PIVOT, queries recursivas complexas).

Resumo da Ópera:

O **CQRS** é sobre separar os modelos (DTO vs Entidade). Se você preenche esse DTO usando LINQ ou SQL, é um detalhe de implementação. Comece com LINQ (.Select). Se ficar lento (o que é raro), otimize para SQL.

🛠️ **Exercício Prático Final: O Dropdown (ComboBox) Otimizado**

O Cenário: A equipe de Front-end está desenvolvendo a tela de **"Cadastro de Novo Veículo"**. Nessa tela, existe um campo de seleção (Dropdown/ComboBox) onde o usuário precisa escolher o **Modelo** do carro.

Porém, o Front-end reclamou: *"Se vocês mandarem apenas o nome do modelo (ex: 'Gol', 'Ka', 'City'), o usuário pode se confundir. Precisamos que o texto do Dropdown venha formatado com a Marca junto, no padrão: **'Marca - Modelo'** (ex: 'Volkswagen - Gol', 'Ford - Ka'). Além disso, a rota tem que ser muito rápida, pois o sistema tem milhares de modelos."*

Sua Missão: Criar uma consulta otimizada de leitura (Query) que entregue exatamente o que o componente de tela precisa, formatando o texto diretamente no banco de dados para poupar processamento da API.

Passo a Passo sugerido:

1. **Crie o DTO de Leitura (Para a Tela):**
    - Crie uma classe chamada ModeloDropdownDTO na pasta de DTOs da Application.
    - Ela deve ter apenas duas propriedades: Id (int) e NomeExibicao (string).
2. **Atualize a Interface:**
    - Crie a interface IModeloQuery (ou adicione na query existente) o método Task<IEnumerable<ModeloDropdownDTO>> ObterModelosParaDropdown().
3. **A Implementação (O Desafio da Concatenação):**
    - Você deve fazer um JOIN entre as tabelas de Modelos e Marcas, e juntar os dois nomes em uma única string (NomeExibicao).
    - *Opção A (LINQ Projection):* O jeito mais fácil e seguro. Use o .Select() e faça a concatenação direto no C#. O EF Core traduzirá isso para o banco.
    - *Opção B (SQL Puro):* Use o SqlQuery. (Atenção à dica abaixo sobre o Postgres!).
4. **A Controller:**
    - Crie um endpoint [HttpGet("dropdown")] na ModeloController (ou VeiculoController), chamando a sua Query.